

GRAPHS × OPEN KNOWLEDGE FORMAT

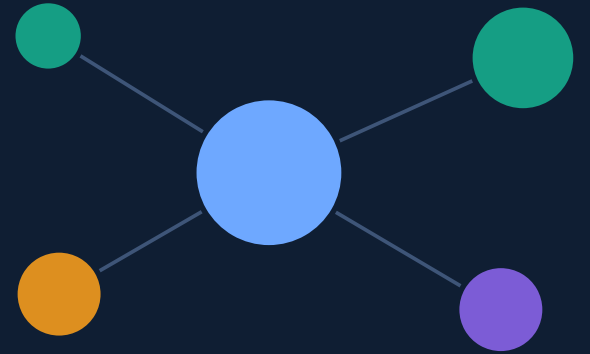
From Markdown to Knowledge Graph

Ingesting Google's Open Knowledge Format into Neo4j — and making it earn its keep with GraphRAG

OKF spec v0.2

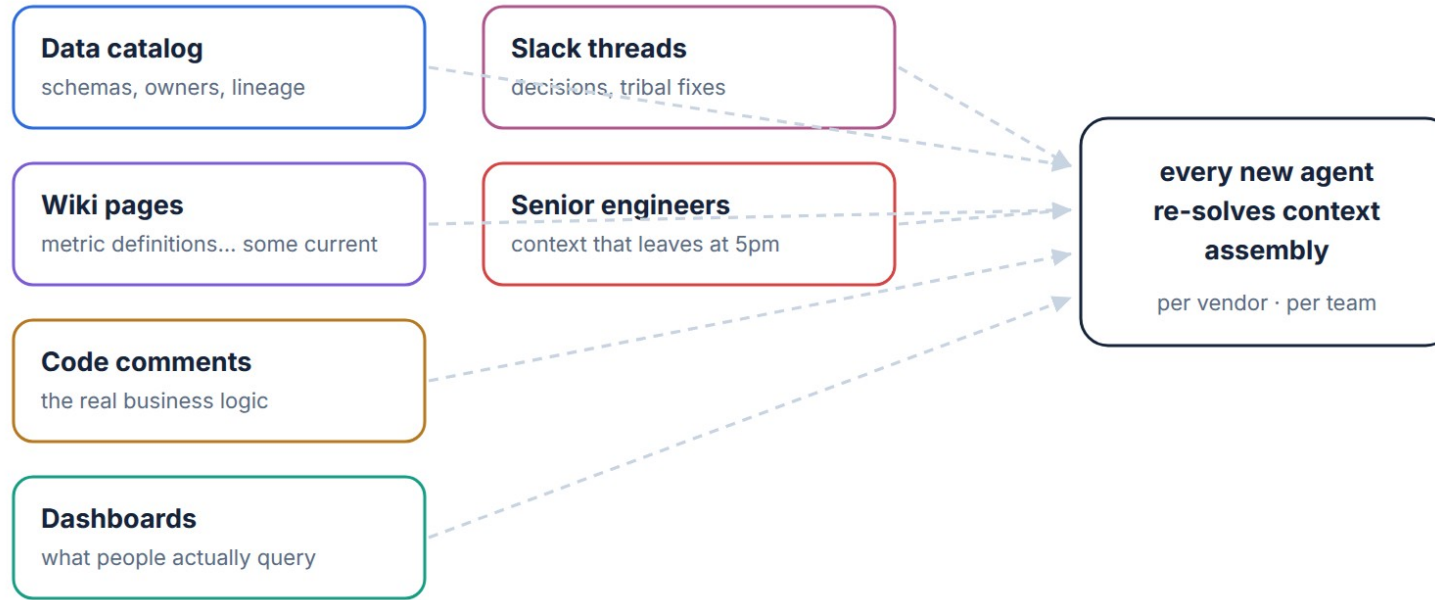
Neo4j 2025.x

neo4j-graphrag 1.18



Why Google shipped a knowledge format at all

the knowledge agents need exists — as five different exhausts



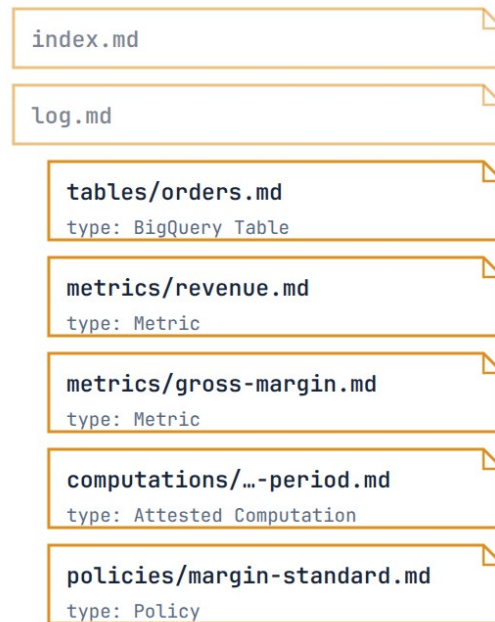
the result

- ✗ no common format
- ✗ locked into catalogs & wikis
- ✗ goes stale silently
- ✗ not portable across orgs

OKF's bet: make knowledge a portable format, not a platform — markdown + frontmatter + links.

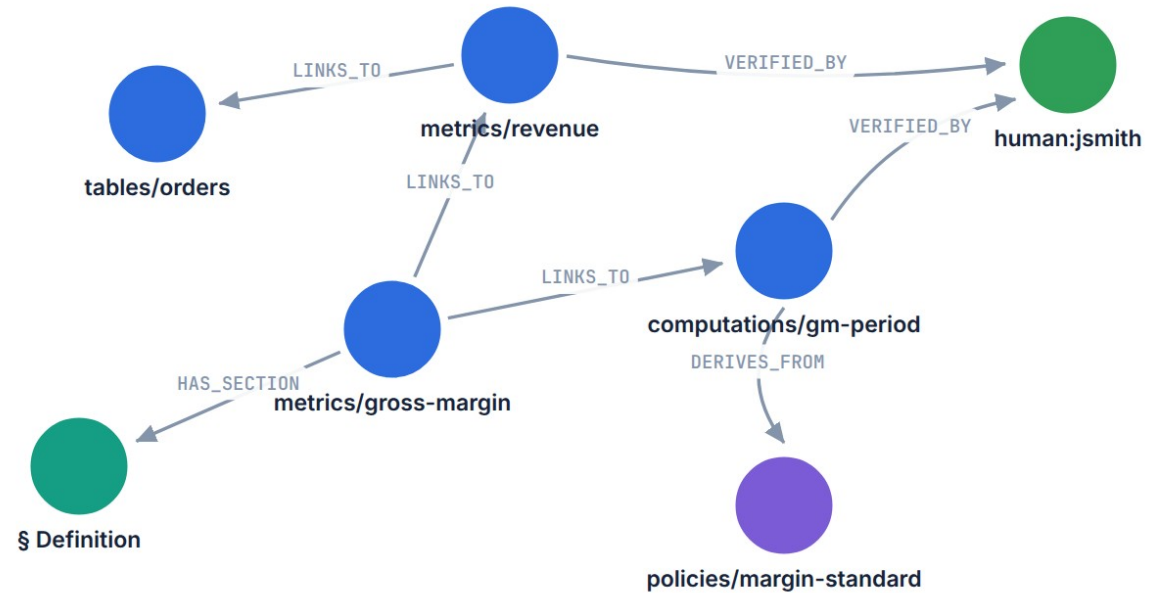
The whole talk in one picture

an OKF bundle



deterministic

no LLM



the same knowledge — traversable, indexable, retrievable

files in git → deterministic parse → property graph → Cypher + GraphRAG

OKF in five facts

github.com/GoogleCloudPlatform/knowledge-catalog · Apache-2.0 · spec v0.2

1 A bundle is a directory of markdown files — that's the whole runtime

Concepts (tables, metrics, playbooks, policies...) as files; ship via git, tarball, or a subdir.

2 One required key: type — everything else is optional families

No schema registry, no central authority. Consumers MUST tolerate unknown types and keys.

3 Path = identity · markdown links = relationships

metrics/gross-margin.md is the id; links carry meaning in surrounding prose. Broken links are legal: not-yet-written knowledge.

4 v0.2 makes trust first-class: sources, generated / verified, status, stale_after

Built for corpora that agents maintain — provenance, trust tiers, freshness, attested computations.

5 Format, not platform — producers and consumers swap independently

Reference producer agent + static graph viewer ship with the repo; GCP Knowledge Catalog ingests OKF.

Anatomy of a bundle

unit of distribution: a git repo, a tarball, a subdirectory — files all the way down

▸ acme_retail/	
▢ index.md	progressive disclosure — what's here, one level at a time
▢ log.md	chronological change history, newest first
▸ tables/	
▪ orders.md	type: BigQuery Table
▸ metrics/	
▪ revenue.md	type: Metric
▪ gross-margin.md	type: Metric
▪ gross-margin-legacy.md	type: Metric · status: deprecated
▸ computations/	
▪ revenue-ytd.md	type: Attested Computation
▪ gross-margin-period.md	type: Attested Computation
▸ policies/	
▪ margin-standard.md	type: Policy
▪ skills/run-on-bq.md	type: Skill — executor instructions
⟨> attesters/sql_equality.py	deterministic receipt checker (not a concept)

structural rules

— path = identity

concept id is the file path sans .md;
moving a file is a rename

— reserved names

index.md and log.md are never
concepts (SPEC §3.1)

— dirs = grouping only

hierarchy is organizational;
meaning lives in the links

— anything extra rides along

code files, viz.html — referenced
by concepts, shipped in the bundle

Anatomy of a concept

metrics/gross-margin.md, dissected — every family is optional except type

```

---
type: Metric
title: Gross Margin
description: Gross margin for a period, per...
tags: [finance, margin, headline-metric]
generated: {by: reference_agent/gemini-2.5-pro,
at: 2026-06-30T14:00:00Z}
verified:
- {by: human:jsmith@acme, at: 2026-07-01...}
status: stable
stale_after: 2026-12-31
sources:
- id: margin-standard
resource: policies/margin-standard.md
author: human:jsmith@acme
last_modified: 2026-06-15
---
# Definition
Gross margin equals [Revenue](./revenue.md)
minus full COGS ... [^margin-standard]
The sanctioned computation is
[computations/gross-margin-period.md](../...)
# What changed in FY2026
Prior to 2026-02-01 ... see
[gross-margin-legacy](./gross-margin-legacy.md)

```

the ONLY required key (§4.1)

unregistered, producer-defined; consumers route on it
→ we mint the :Metric secondary label from it

who wrote it — agent, human, process (§7)

actor convention keys the trust machinery
→ (:Actor {kind}) + [:GENERATED_BY {at}]

who confirmed it, independently (§5.2)

generation ≠ verification
→ [:VERIFIED_BY {at}] edges → trust tier

lifecycle (§5.4–5.5)

deprecated is kept, not deleted; staleness is a
date comparison → query-time verdicts

provenance with credibility signals (§5.1)

author / usage_count / last_modified per source
→ (:Source) node + [:DERIVES_FROM] edge

conventional # sections (§4.2)

Schema, Computation, Examples...
→ (:Section) nodes = natural retrieval units

footnote [^id] = claim-level citation

label joins to sources[.id] — survives reordering
→ (:Section)-[:CITES]→(:Source)

markdown links = edges

relationship kind lives in surrounding prose
→ [:LINKS_TO {section}] keeps that context

The v0.2 trust machine

four optional frontmatter families — absence is meaningful, never fatal (SPEC §5, §10)

provenance

sources: — what it derives from

```
resource:      URL · path · scope
author:        team:finance-fpa
usage_count:   5000 ← liveness
last_modified: 2026-06-15
```

signals, not scores —

credibility is inferred by the consumer, never stored

footnotes [^id] attribute individual claims to sources

trust

who wrote it ≠ who confirmed it

```
generated:
  {by: agent/..., at: ...}
```

```
verified:
  [{by: human:..., at: ...}]
```

⇒ derived trust tier



advisory, not access control

lifecycle

is it current? a date comparison



absent status ⇒ stable



deprecated is preserved, not deleted — history stays linkable

absolute dates, no TTL math

attestation (§10)

was the number produced the sanctioned way? per-run, runtime

Attested Computation

SQL + typed parameters

executor → receipt

job_id, executed_sql

attester (no LLM)

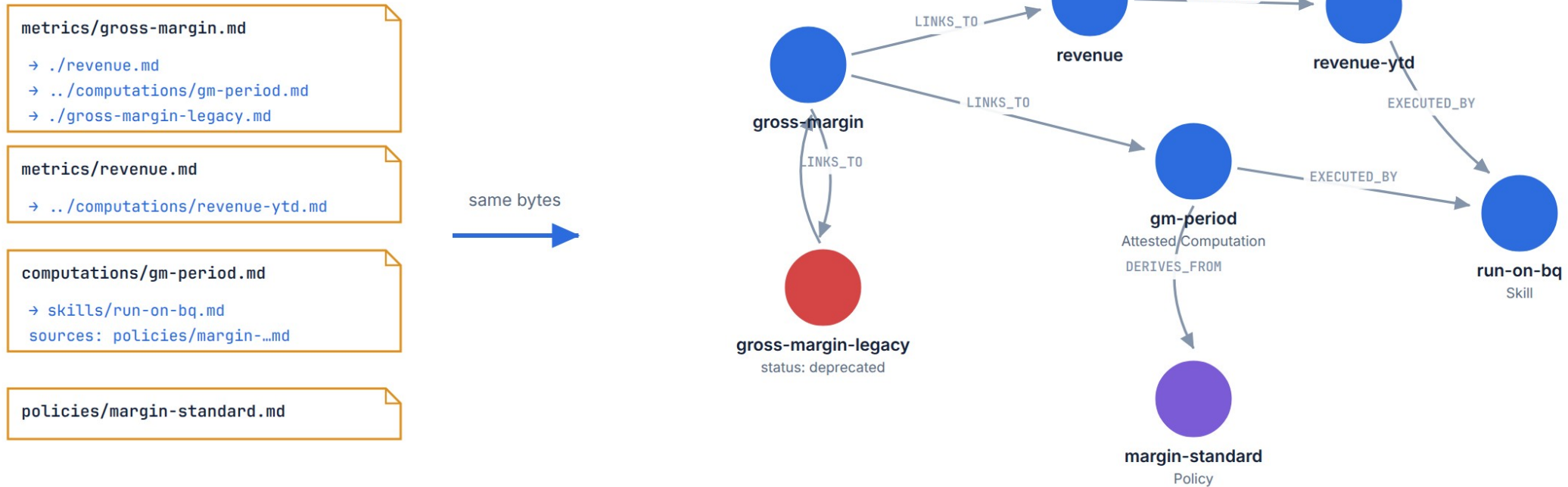
deterministic verdict

agents may fill parameters; they may NOT edit the SQL.

verified ≠ attested: doc-level vs per-run — you need both

A bundle is already a graph

Google's own framing: “graph-shaped, not just tree-shaped” — every bundle even ships a static viz.html



nothing is missing from the files view — what's missing is the tooling: traversal, indexes, aggregation, retrieval

Why materialize it in a database

viz.html browses one bundle beautifully — everything on the right is a database job

viz.html (ships in every bundle)

static, self-contained, zero-backend — great for browsing

- ✓ render one bundle
- ✓ click through links
- ✓ read a concept
- ✗ filter by trust tier / staleness
- ✗ impact analysis across hops
- ✗ aggregate over frontmatter
- ✗ vector / hybrid retrieval
- ✗ join multiple bundles

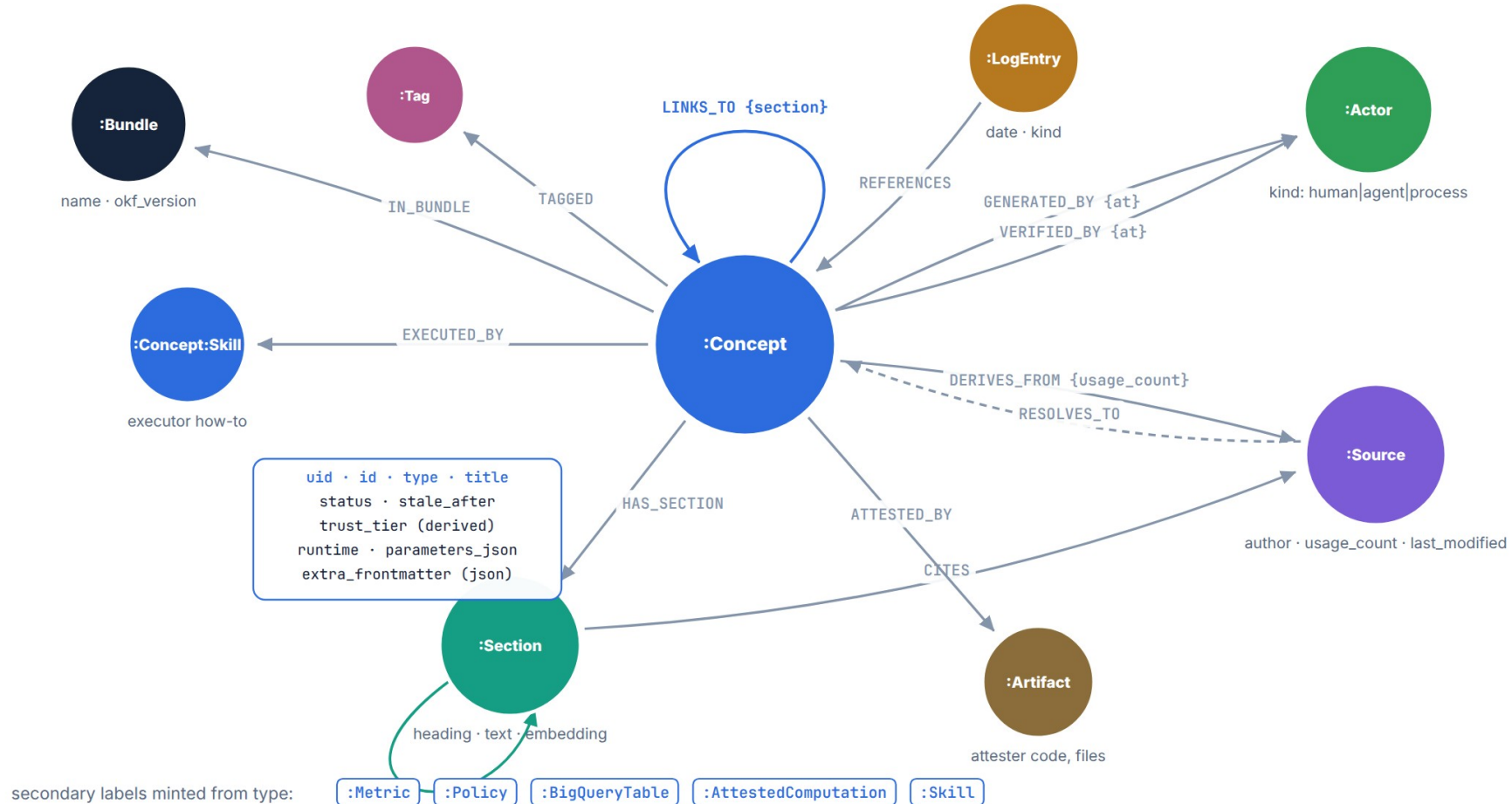
Neo4j property graph

the same structure, plus a query planner, indexes and drivers

- ✓ everything on the left
- ✓ Cypher: traversals, aggregations, paths
- ✓ secondary labels from OKF types
- ✓ vector + fulltext indexes on sections
- ✓ GraphRAG retrievers (neo4j-graphrag)
- ✓ multi-bundle, one graph
- ✓ constraints: uid uniqueness, idempotent sync
- ✓ GDS when you outgrow plain Cypher

The graph model

structural + provenance + lexical layers in one property graph — plain Cypher (APOC only in the optional extraction appendix)



also: `(:Section)-[:MENTIONS]→(:Concept) · vector index section_embeddings + fulltext section_fulltext on (:Section)`

The mapping, construct by construct

OKF (SPEC v0.2)

PROPERTY GRAPH

bundle directory	→ (:Bundle {name, okf_version})	
concept file path/x.md	→ (:Concept {uid, id, status, trust_tier, ...})	
frontmatter type:	→ secondary label :Metric :Policy :AttestedComputation	only required key → cheap routing
markdown link	→ [:LINKS_TO {section, text}]	section context preserved on the edge
broken link	→ stub (:Concept {stub: true})	not-yet-written knowledge, queryable
sources[] entry	→ (:Source {author, usage_count, ...}) + [:DERIVES_FROM]	intrinsic props on node · declaration props on edge
source resource → internal path	→ (:Source)-[:RESOLVES_TO]→(:Concept)	provenance chains become traversable
generated / verified	→ (:Actor {kind}) + [:GENERATED_BY / :VERIFIED_BY {at}]	trust tier derived from verifier kinds
body # sections	→ (:Section {heading, text, embedding}) + [:NEXT]	conventional headings = chunking for free
footnote [^id]	→ (:Section)-[:CITES]→(:Source)	claim-level provenance
executor / attester	→ [:EXECUTED_BY]→(:Skill) · [:ATTESTED_BY]→(:Artifact)	
tags	→ (:Tag) + [:TAGGED]	
log.md entries	→ (:LogEntry {date, kind})-[:REFERENCES]→(:Concept)	change history joins the graph
index.md	→ not ingested – derivable; okf_version lifted to Bundle	progressive disclosure is a serving concern

Five decisions that earn their keep later

1 Sections, not fixed-size chunks

h1 headings are semantic retrieval units (fence-aware splitting; long sections split at paragraphs).

2 Type strings → secondary labels

“BigQuery Table” → :BigQueryTable — label-scoped Cypher + index locality, original kept in c.type.

3 Trust tier: derived AND materialized

The CASE derivation is the living definition; the property is the cheap filter. Both shown right.

4 Unknown frontmatter survives

Spec: consumers MUST NOT reject; SHOULD round-trip. Extra keys → extra_frontmatter JSON (acme ships a not: field!).

5 Ingestion is idempotent

MERGE on uid everywhere; re-running a bundle is a sync, not a duplicate. reset(bundle) scopes deletes.

```

– trust tier, derived live (SPEC §5.3)
MATCH (c:Concept)
OPTIONAL MATCH (c)-[v:VERIFIED_BY]->(a:Actor)
WITH c, collect(a.kind) AS kinds
RETURN CASE
  WHEN size(kinds) = 0 THEN 'unverified'
  WHEN 'human' IN kinds THEN 'human-reviewed'
  ELSE 'machine-confirmed'
END AS trust_tier, count(*) AS n

```

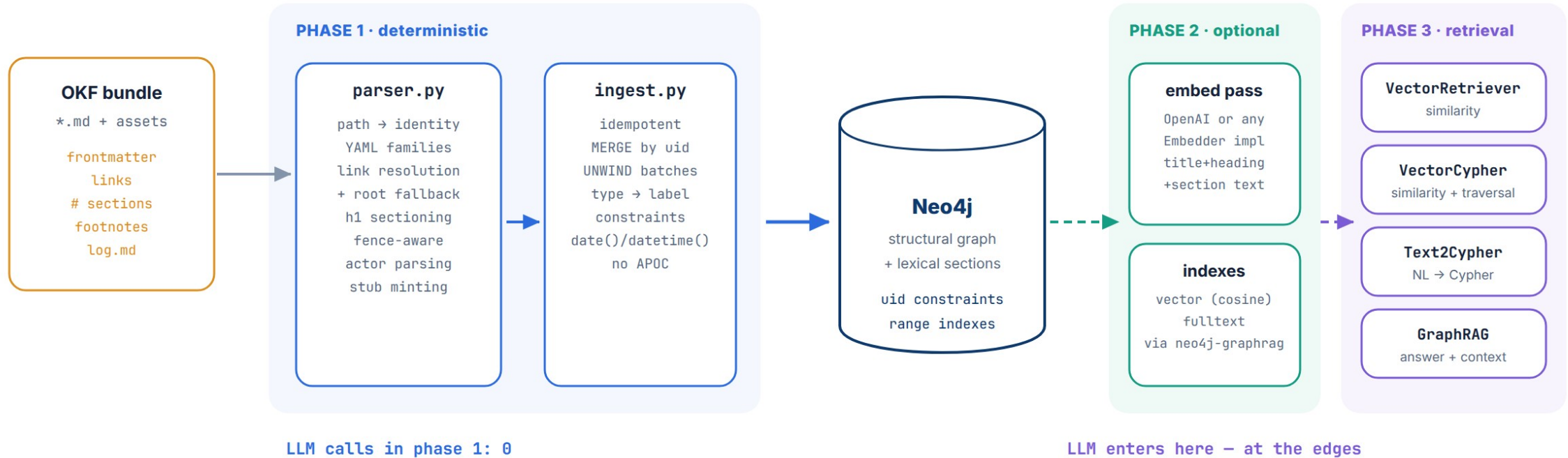
```

– staleness is a date comparison (§5.5)
WHERE c.stale_after IS NOT NULL
  AND date() >= c.stale_after
// absolute date – no TTL arithmetic

```

Ingestion architecture

phase 1 is pure structure — an LLM never touches it; embeddings are a separate, optional pass



The graph pays rent before any AI shows up

```

– can we serve it? one query, whole estate
MATCH (c:Concept) WHERE NOT c.stub
RETURN c.id, c.trust_tier,
  CASE WHEN c.status = 'deprecated'
    THEN 'do not serve'
    WHEN date() >= c.stale_after
    THEN 're-verify first'
    ELSE 'servable' END AS verdict

metrics/gross-margin-legacy → do not serve
skills/run-on-bq (unverified) → servable

```

```

– blast radius: what breaks if tables/orders changes?
MATCH p = (up:Concept)-[:LINKS_TO|EXECUTED_BY|
  DERIVES_FROM|RESOLVES_TO|HAS_SECTION|MENTIONS
  *1..6]->(t:Concept {id:'tables/orders'})
RETURN up.id, min(length(p)) AS hops // via :Source too

h1 policies/revenue-recognition
h2 computations/revenue-ytd (provenance chain)
h3 metrics/revenue · gross-margin · gm-period
h4 gross-margin-legacy · margin-standard

```

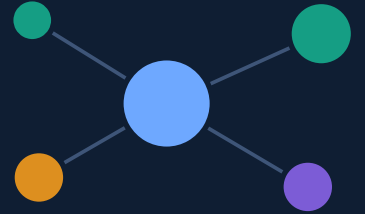
co-citation concepts whose CLAIMS share a source – footnote-level, via (:Section)-[:CITES]->(:Source)

authoring backlog MATCH (ghost:Concept {stub:true})<-[:LINKS_TO]-(c) – every not-yet-written doc, with who wants it

load-bearing knowledge incoming-degree ranking (or PageRank via GDS) – what to re-verify first when the quarter turns

Ingest and interrogate

notebooks/okf_graphrag_demo.ipynb — sections 1–4 (no API key needed)



1

`docker compose up -d && uv sync`

Neo4j 2025.x community · uv installs deps + the okf-graph CLI

2

`uv run okf-graph ingest bundles/acme_retail --reset`

parse → 9 concepts · 22 sections · 15 links · 4 log entries

3

`trust tiers · staleness · impact · co-citation`

the four governance queries, live on the real bundle

4

`neo4j-viz – color by status, spot the deprecated island`

interactive graph inside the notebook

Three retrieval patterns, one graph

pick per question, compose freely — GraphRAG wraps any retriever with any LLM

VectorRetriever

semantic similarity, k-NN on the vector index

"how do we calculate gross margin?"

embed(query)

section_embeddings

#1 legacy definition ▲ deprecated
#2 current definition
#3 FY2026 change note

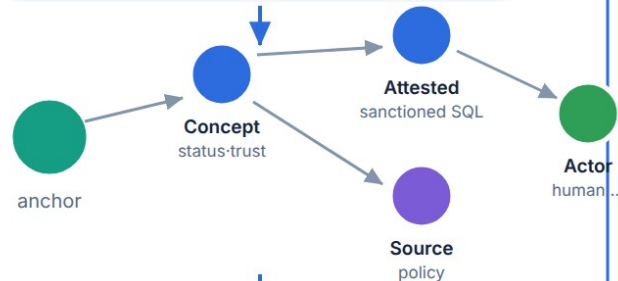
top-k section texts → LLM

- ✓ fast, zero modeling
- × blind to status, trust, links

VectorCypherRetriever

vector anchor → Cypher traversal per hit

same question



status: deprecated → SUPERSEDED BY ...
trust: human-reviewed (jsmith, 07-01)
sanctioned SQL + freshness verdict

governed context → LLM

- ✓ the graph decides what the LLM sees

Text2CypherRetriever

no similarity anchor — compile NL to Cypher

"which metrics were never human-reviewed?"

LLM + schema + examples

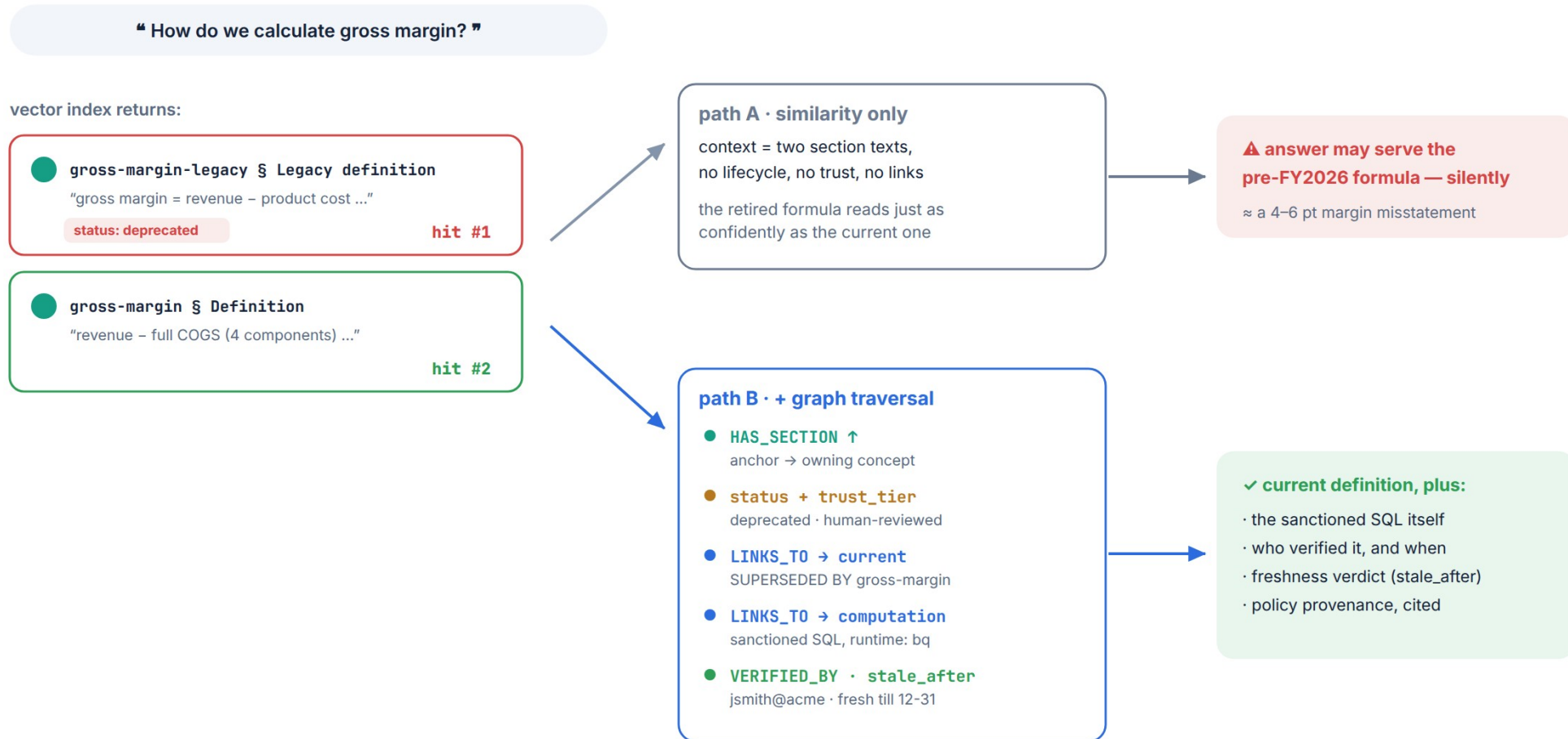
```
MATCH (m:Metric)
WHERE m.trust_tier <>
'human-reviewed'
RETURN m.id, m.trust_tier
```

rows → answer (with the query shown)

- ✓ aggregates, negation, paths
- ✓ auditable — the Cypher is the receipt
- ▲ run it with a read-only user

The governance trap, step by step

the deprecated definition is the best-written text about the question — similarity is doing its job



same model, same embeddings, same top-k — the only variable is what retrieval hands the LLM. (ranking: recorded rehearsal run)

The governed retrieval_query

okf_graph/queries.py · GOVERNED_RETRIEVAL_QUERY — runs after the ANN lookup, per anchor

```

– python: swap ONE argument
from neo4j_graphrag.retrievers import \
    VectorRetriever, VectorCypherRetriever
from neo4j_graphrag.generation import GraphRAG
from neo4j_graphrag.llm import OpenAILLM

baseline = VectorRetriever(driver,
    'section_embeddings', embedder)

governed = VectorCypherRetriever(
    driver, 'section_embeddings',
    retrieval_query=GOVERNED_RETRIEVAL_QUERY,
    embedder=embedder)

rag = GraphRAG(retriever=governed,
    llm=OpenAILLM('gpt-4o'))
rag.search('how do we calculate
    gross margin?', return_context=True)

```

```

– cypher: what each anchor pulls in
WITH node, score // ANN anchor
MATCH (c)-[:HAS_SECTION]->(node)
// lifecycle + who verified
OPTIONAL MATCH (c)-[v:VERIFIED_BY]->(va)
// sanctioned SQL, ≤2 link-hops away
OPTIONAL MATCH p=(c)-[:LINKS_TO*1..2]->
    (ac:AttestedComputation) ...
// provenance + the current replacement
OPTIONAL MATCH (c)-[:DERIVES_FROM]->(src)
OPTIONAL MATCH (c)-[:LINKS_TO]->(current)
    WHERE c.status = 'deprecated' ...
RETURN '=== '+c.title+' ===\n'+
    'status: ... SUPERSEDED BY ...'+
    'trust/freshness verdicts'+node.text+
    'sanctioned SQL...' AS info, score

```

The 2-hop LINKS_TO is what lets a deprecated anchor still reach the sanctioned SQL — through its replacement.

Text2Cypher: questions with no similarity anchor

“ Which concepts are stale or deprecated, and what replaces them? ”

```
MATCH (c:Concept) WHERE c.status='deprecated' OR date() >= c.stale_after
OPTIONAL MATCH (c)-[:LINKS_TO]->(r {status:'stable'}) RETURN c.id, r.id
```

“ Which metrics were never reviewed by a human? ”

```
MATCH (m:Metric) WHERE m.trust_tier <> 'human-reviewed'
RETURN m.id, m.trust_tier
```

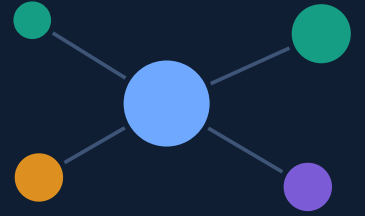
“ Who verified revenue, and which sources back it? ”

```
MATCH (c {id:'metrics/revenue'})-[v:VERIFIED_BY]->(a)
OPTIONAL MATCH (c)-[:DERIVES_FROM]->(s) RETURN a.id, v.at, collect(s.title)
```

Guardrails: hand-written schema string + few-shot examples (in queries.py) beat a dumped SHOW SCHEMA · run with a read-only user · generated Cypher explores, canned Cypher ships to dashboards

GraphRAG on the bundle

notebook sections 5–8 — needs OPENAI_API_KEY (embeddings + gpt-4o)



1

embed 22 sections · create vector + fulltext indexes

okf_graph embed pass → neo4j-graphrag create_vector_index

2

ASK: “How do we calculate gross margin, exactly?”

baseline VectorRetriever — watch the deprecated section outrank the current one

3

same question, VectorCypherRetriever

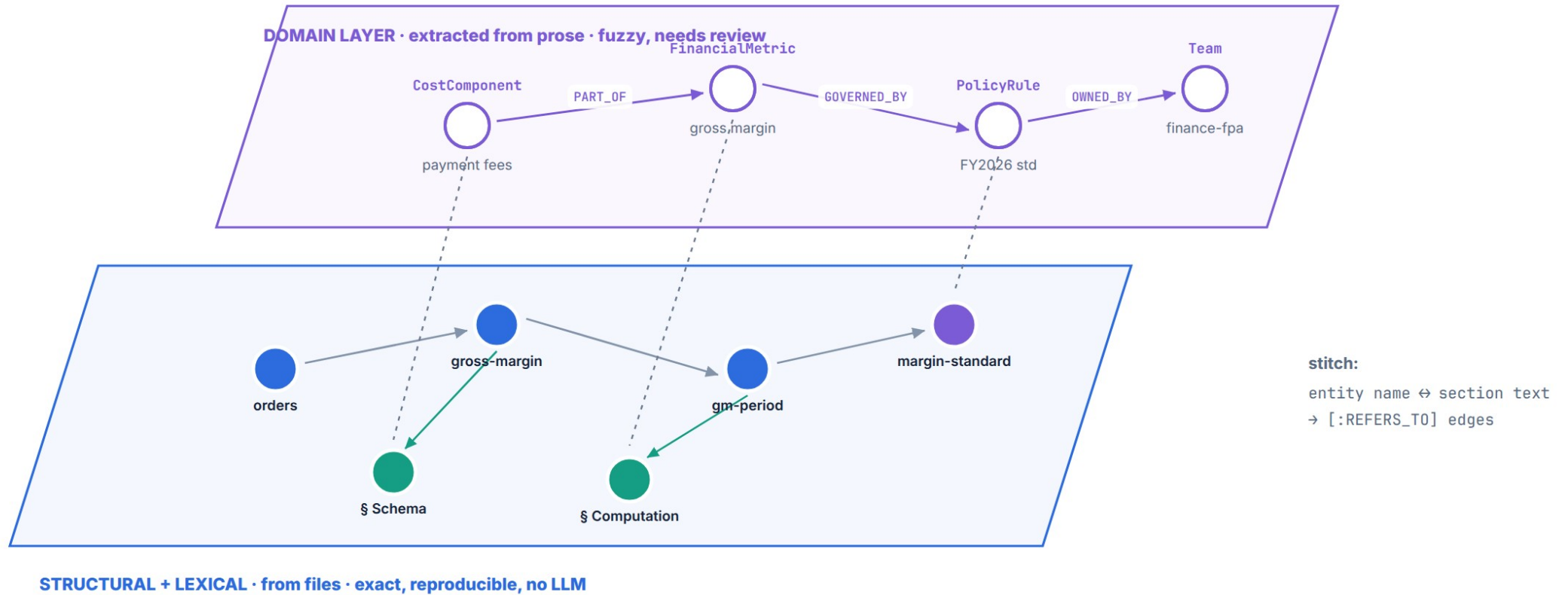
governed context: SUPERSEDED BY · trust · freshness · sanctioned SQL — answer cites policy

4

Text2Cypher: “what breaks if orders changes?”

generated Cypher shown, rows returned — no embeddings involved

Deterministic structure + LLM extraction, one graph



SimpleKGPipeline (same package) extracts *CostComponent* / *PolicyRule* / *Team* entities from the policy prose — connecting concepts that never explicitly link.

What to take home

For your data estate

- OKF is a low-friction interchange: catalogs, wikis, agents all write files
- One Cypher layer turns those files into serve/no-serve verdicts, impact analysis, provenance chains
- Multi-bundle by construction — bundle is just a property

For your RAG stack

- Chunking on authored sections beats fixed windows — the format hands it to you
- Similarity $\neq$ governance: anchor with vectors, assemble context with the graph
- One package (neo4j-graphrag): indexes, 3 retriever families, GraphRAG, KG builder

For your agents

- Trust tiers + stale_after become retrieval-time FILTERS, not documentation
- Attested computations: agents bind parameters, never author SQL
- The graph is the context service OKF says every org keeps rebuilding

Thanks — go materialize your knowledge.

demo repo (this talk, runnable) github.com/johnymontana/neo4j-okf

OKF spec v0.2 + sample bundles github.com/GoogleCloudPlatform/knowledge-catalog

neo4j-graphrag package docs neo4j.com/docs/neo4j-graphrag-python

GraphRAG retriever patterns neo4j.com/developer/genai-ecosystem

Will Lyon · questions welcome

